

Project 7 of 15

Gym & Fitness Club Management

Domain: FitTech | Spark to Code 2026 — Final Capstone Project

This is your Spark to Code 2026 capstone project. It builds directly on everything covered in the bootcamp — ERD & mapping, C# models, EF Core DbContext, Web API controllers with LINQ, and a Bootstrap-based frontend — and adds two new pieces you are required to learn on your own: **JWT authentication** and a basic **email-sending service**. Use official Microsoft docs and the resources shared in class as your starting point for the self-study parts.

Business idea: A gym-chain system managing branches, membership plans, class schedules and bookings, trainers, attendance and member progress.

1. Suggested Core Entities (minimum 12)

Use this as a starting point for your ERD design session — you may rename, merge, or add entities as long as you keep at least 12 models including User.

1. User (Member/Trainer/Admin, used for JWT)	7. Equipment
2. Trainer Profile	8. Payment
3. MembershipPlan	9. Attendance
4. Membership	10. WorkoutPlan
5. ClassSchedule	11. BodyMetric
6. ClassBooking	12. Branch

2. Functional Requirements

- **Minimum 12 models**, including the **User** model, which is the basis for JWT authentication (Id, Name, Email, PasswordHash, Role, etc.).
- **Full ERD & mapping** for all entities before writing any code — including primary keys, foreign keys, and one-to-many / many-to-many relationships where applicable.
- **EF Core Code-First**, with C# model classes matching the ERD and a single DbContext registering all DbSet.
- **JWT Authentication (self-study)**: Register/Login endpoints, password hashing, token generation, and [Authorize] applied to protect the sensitive endpoints across the project.
- **Email Service (self-study)**: a basic email-sending service (e.g. using MailKit / SMTP) triggered from at least one meaningful action in the project — see the domain-specific hint below.

Domain-specific email trigger: send a membership-activation email on sign-up, and a class-booking confirmation email.

Controller coverage — 8 cases per controller (min.)

Every one of the 12+ models must have its own Web API controller, injected with DbContext directly from Program.cs (as covered in class). Each controller must implement **at least 8 usage cases**, distributed as follows. GET cases are encouraged to reach across relationships into other models/tables in the same DbContext, not only their own table.

#	Case	Requirement
1	POST	Create a new record (with model validation / DTO).
2	PUT / PATCH	Update a record (full or partial update).
3	PUT / PATCH	A second, distinct update case (e.g. status change, or update via a related entity's FK).

#	Case	Requirement
4	DELETE	Delete a record (consider soft-delete if relevant).
5	GET (list)	Get all records, including at least one related entity (navigation property) via Include().
6	GET (find)	Get a single record by Id / key.
7	GET (filter)	Filter records using LINQ (Where) on at least one meaningful field, e.g. by date range, status, category, or a related table's property.
8	GET (sort/aggregate)	Sort (OrderBy/OrderByDescending) and/or aggregate (Count, Sum, Average, GroupBy) records.

Frontend

Build the pages needed to fully exercise every controller: HTML/CSS/JavaScript with Bootstrap for layout and components, calling the Web API endpoints for CRUD. Include the login/registration screens for the JWT flow, and make sure the JWT is attached to protected requests.

3. Team Structure — 6 Developers

The team has **6 developers**. Split the 12 models into 6 pairs — each developer owns 2 models end-to-end: ERD contribution, C# model + DbContext registration, their controller (8 cases each = 16 endpoints per developer), Swagger/Postman testing, and later the frontend pages for their 2 models. A suggested (not mandatory) split is shown below — the team lead should confirm the final assignment during the Sprint 1 kickoff.

Developer	Assigned Models & Controllers
Developer 1	User (Member/Trainer/Admin, used for JWT) & Trainer Profile
Developer 2	MembershipPlan & Membership
Developer 3	ClassSchedule & ClassBooking
Developer 4	Equipment & Payment
Developer 5	Attendance & WorkoutPlan
Developer 6	BodyMetric & Branch

4. Sprint Plan (3 Weeks)

Week	Sprint	Activities
Week 1	Sprint 1 — Backend	1) Team designs the ERD & table mapping together. 2) Team lead performs the initial solution/project setup (solution structure, DbContext, base Program.cs, Git repo & branching). 3) Each developer implements their 2 assigned models + DbContext registration + their 2 controllers (8 cases each). 4) Each developer self-tests their endpoints via Swagger & Postman. 5) Sprint closes with a full backend/system test: the team collaboratively tests every controller end-to-end, including cross-model scenarios.
Week 2	Sprint 2 — Frontend	1) Team designs the frontend pages needed to serve all controllers. 2) Each developer implements the pages for their own 2 models (HTML/CSS/JS + Bootstrap, calling the Week 1 APIs). Self-study additions (JWT login/register screen, protected-route handling, email-triggered actions) are integrated here. 3) Pages are integrated with the tested APIs. 4) Sprint closes with full user (system) testing of the integrated application.
Week 3	Project Demo	Each team demonstrates the full system live: authentication flow, all 12+ models end-to-end (CRUD + filtering/sorting/aggregation), the email notification feature, and the complete frontend user journey.

5. Deliverables

- Full ERD diagram (image or tool export) and a short mapping note (PK/FK, relationship types).
- Source code in a shared GitHub repository, organized by the agreed solution/project structure, with one branch per developer merged via pull requests.
- Swagger documentation reachable at run time, and a Postman collection covering all 8+ cases per controller.
- Working JWT authentication protecting the required endpoints, and a working email notification for the domain-specific trigger described above.
- A functioning frontend that performs full CRUD against every controller through the UI, using Bootstrap.
- A short README explaining how to run the project (backend + frontend), the final ERD, and each developer's responsibility.

6. Evaluation Criteria

- Correctness & completeness of the ERD and its EF Core mapping.
- Each controller meets the 8-case minimum with correct LINQ usage (filtering, sorting, aggregation, and use of related entities).
- JWT authentication works end-to-end and protects the intended endpoints.
- Email notification actually sends for the specified domain trigger.
- Frontend fully exercises the API (no missing CRUD flows) and uses Bootstrap consistently.
- Git history shows real, distributed contribution from all 6 developers.
- Quality of the Sprint 1 system test and the Sprint 2 user test (evidence expected, e.g. a short test log or checklist).
- Clarity and confidence of the Week 3 live demo.